

2장(사이킷런으로 시작하는 머신러닝) + 3장(평가)

01. 사이킷런(scikit-learn)

- 파이썬 머신러닝 라이브러리 중 가장 많이 사용되는 라이브러리
- 가장 쉽고 효율적인 개발 라이브러리 제공
- 최근 딥러닝 전문 라이브러리들(텐서플로, 케라스)등의 인기 속에도 여전히 대표적인 파이썬 ML 라이브러리
- 특징
 - 쉽고 가장 파이썬스러운 API제공
 - 머신러닝을 위한 다양한 알고리즘과 개발을 위한 편리한 프레임워크와 API제공
 - 많은 환경에서 사용됨
- 설치
 - Anaconda 설치하면 기본으로 설치 완료됨
 - 버전 확인

```
import sklearn
print(sklearn.__version__)
```

02. 머신러닝 만들어 보기 - 불꽃 품종 예측하기

문제 설명: 불꽃 데이터 세트로 불꽃의 품종을 분류(Classification)하기

- 불꽃 데이터 세트는
 - 피처(Feature) :
 - 꽃잎의 길이 (petal length)
 - 꽃잎의 너비 (petal width)
 - 꽃받침의 길이 (sepal length)
 - 꽃받침의 너비 (sepal width)

- 레이블(Label) : 붓꽃의 품종
 - Setosa (0)
 - Vesicolor (1)
 - Virginica (2)
- 분류(Classification) : 대표적인 지도학습(Supervised Learning)방법중 하나
 - 명확한 정답이 주어진 학습 데이터(다양한 피쳐와 분류 결정값인 레이블 데이터)로 모델을 학습한뒤, 별도의 테스트 데이터 세트에서 미지의 레이블을 예측하는 방식
 - 학습 데이터 세트: 학습을 위해 주어진 데이터 세트
 - 테스트 데이터 세트: 머신러닝 모델의 예측 성능을 평가하기 위해 별도로 주어진 데이터 세트
- 사이킷런
 - 사이킷런 패키지내의 모듈들(sklearn으로 시작)
 - sklearn.datasets : 사이킷런에서 자체적으로 제공하는 데이터 세트를 생성하는 모듈 모임
 - sklearn.tree : 트리 기반 ML 알고리즘을 구현한 클래스의 모임
 - sklearn.model_selection : 학습 데이터와 검증 데이터, 예측 데이터로 데이터를 분리하거나 최적의 하이퍼 파라미터로 평가하기 위한 모듈 모임
 - 하이퍼 파라미터 : 머신러닝 알고리즘별로 최적의 학습을 위해 직접 입력하는 파라미터들, 머신러닝 알고리즘의 성능을 튜닝 할 수 있음
 - load_iris() 이용해서 붓꽃 데이터 세트 생성
 - 사용한 알고리즘 : 의사 결정 트리(Decision Tree)알고리즘
 - DecisionTreeClassifier 적용

주요 단계:

1. 데이터 세트 분리
2. 모델 학습
3. 예측 수행
4. 평가

한 단계씩 자세히 풀어보겠습니다.

0. 준비

붓꽃 데이터 세트 생성:

```
iris = load_iris()
```

1. 데이터 세트 분리

```
X_train, X_test, y_train, y_test = train_test_split(iris_data, iris_label,  
test_size=0.2, random_state=11)
```

`train_test_split()` : 사이킷런이 제공한 API

- Parameters:
 - feature data set(`iris_data`)
 - label data set(`iris label`)
 - test size : 학습 데이터와 테스트 데이터의 비율
 - 0.2 → 학습 데이터 80%, 테스트 데이터 20%
 - `random_state` : 호출할 때마다 같은 학습/테스트 용 데이터 세트를 생성하기 위해 주어지는 난수 발생 값
- `X_train` : 학습용 피쳐 데이터 세트
- `X_test` : 테스트용 피쳐 데이터 세트
- `y_train` : 학습용 레이블 데이터 세트
- `y_test` : 테스트용 레이블 데이터 세트

2. 모델 학습

```
dt_clf = DecisionTreeClassifier(random_state=11)  
dt_clf.fit(X_train,y_train)
```

- 사이킷런의 의사 결정 트리 클래스인 `DecisionTreeClassifier`를 객체로 생성
- `fit()` 메서드로 학습 수행

3. 예측 수행

```
pred = dt_clf.predict(X_test)
```

- 학습된 DecisionTreeClassifier객체의 predict() 메서드로 학습된 모델 기반에서 테스트 데이터 세트에 대한 예측값을 반환

4. 평가

```
print(accuracy_score(y_test,pred))
```

- 머신러닝 모델의 성능 평가 방법은 여러가지가 있으나, 여기서는 정확도를 예측
 - 정확도 : 예측 결과(예측한 붓꽃 품종)가 실제 레이블 값(실제 테스트 데이터 세트의 붓꽃 품종)과 얼마나 정확하게 맞는지
 - 사이킷런이 제공하는 accuracy_score()함수로 진행
 - Parameters:
 - 실제 레이블 데이터 세트
 - 예측 레이블 데이터 세트

03. 사이킷런의 기반 프레임워크

- Estimator
 - 지도학습의 주요 두축인 분류(Classification)와 회귀(Regression)의 다양한 알고리즘을 구현한 모든 사이킷런 클래스는 fit()과 predict()을 이용해 학습과 예측 결과를 반환
 - fit() : ML 모델 학습의 위한 메서드
 - predict() : 학습된 모델의 예측을 위한 메서드
 - 분류 알고리즘을 구현한 클래스 : Classifier
 - DecisionTreeClassifier
 - RandomForestClassifier
 - GradientBoostingClassifier
 - GaussianNB
 - SVC
 - 회귀 알고리즘을 구현한 클래스 : Regressor
 - LinearRegression
 - Ridge

- Lasso
- RandomForestRegressor
- GradientBoostingRegressor
- Classifier + Regressor = Estimator
- 비지도학습인 차원 축소, 클러스터링, 피처 추출(Feature Extraction)등을 구현한 클래스도 fit()과 transform 적용
 - fit() : 입력 데이터의 형태에 맞춰 데이터를 변환하기 위한 사전 구조를 맞추는 작업
 - transform() : 입력 데이터의 차원 변환, 클러스터링, 피처 추출 등의 실제 작업
 - fit_transform() : fit() + transform()
- 사이킷런의 주요 모듈
 - 예제 데이터
 - sklearn.datasets : 사이킷런에 내장되어 예제로 제공하는 데이터 세트
 - 피처 처리
 - sklearn.preprocessing : 데이터 전처리에 필요한 다양한 가공 기능
 - sklearn.feature_selection : 알고리즘에 큰 영향 미치는 피처를 우선으로 선택 작업 수행하는 기능들
 - sklearn.feature_extraction : 텍스트 데이터나 이미지 데이터의 벡터화된 피처를 추출하는데 사용
 - sklearn.feature_extraction.text : 텍스트 데이터
 - sklearn.feature_extraction.image : 이미지 데이터
 - 피처 처리 & 차원 축소
 - sklearn.decomposition : 차원 축소 관련한 알고리즘 지원
 - 데이터 분리, 검증 & 파라미터 튜닝
 - sklearn.model_selection : 교차 검증을 위한 학습용/테스트용 분리
 - 평가
 - sklearn.metrics : 분류, 회귀, 클러스터링, 페어와이즈에 대한 다양한 성능 측정 방법
 - ML 알고리즘

- `sklearn.ensemble` : 앙상블 알고리즘
- `sklearn.linear_model` : 선형 회귀, 릿지, 라쏘 및 로지스틱 회귀등 회귀 관련 알고리즘 지원
- `sklearn.naive_bayes` : 나이브 베이즈 알고리즘
- `sklearn.neighbors` : 최근점 이웃 알고리즘
- `sklearn.svm` : 서포트 벡터 머신 알고리즘
- `sklearn.tree` : 의사 결정 트리 알고리즘
- `sklearn.cluster` : 비지도 클러스터링 알고리즘
- 유틸리티
 - `sklearn.pipeline` : 피처 처리 등의 반환과 ML 알고리즘 학습, 예측 등을 함께 묶어서 실행 할 수 있는 유틸리티
- 머신러닝 모델 구축하는 주요 프로세스
 1. 피처의 가공, 변경, 추출을 수행하는 피처 처리(feature processing)
 2. ML 알고리즘 학습/예측 수행
 3. 모델 평가
 - 위 단계들을 반복적으로 수행
- 내장된 예제 데이터 세트들
 - 분류나 회귀 연습용
 - 회귀
 - `datasets.load_boston()`
 - `datasets.load_diabetes()`
 - 분류
 - `datasets.load_breast_cancer()`
 - `datasets.load_digits()`
 - `datasets.load_iris()`
 - 분류와 클러스터링을 위한 표본 데이터 생성기
 - 분류
 - `datasets.make_classifications()`

- datasets.make_blobs()
- 클러스터링
 - 딕셔너리 형태로 되어 있음
 - 주요 키 값
 - data : 피처의 데이터 세트
 - 넘파이 배열(ndarray) 타입
 - 예: `[[5.1 3.5 1.4 0.2]`
`[4.9 3. 1.4 0.2]`
`....`
`]]`
 - target : 분류 시 레이블 값, 회귀일 때는 숫자 결괏값 데이터 세트
 - 넘파이 배열(ndarray) 타입
 - 예:
 - `[0 0 0 0 .... 2 2 2]`
 - target_names : 개별 레이블의 이름
 - 넘파이 배열 또는 파이썬 리스트(list) 타입
 - 예: `['setosa' 'versicolor' 'virginica']`
 - feature_names : 피처의 이름
 - 넘파이 배열(ndarray) 타입
 - 예: `['sepal length', 'sepal width', 'petal length', 'petal width']`
 - DESCR : 데이터 세트에 대한 설명
 - 스트링 타입
- 예제:

```
from sklearn.datasets import load_iris
iris_data = load_iris()
print(type(iris_data)) # sklearn.util.Bunch 클래스 (파이썬 딕셔너리 자료형 유사)
```

```
keys = iris_data.keys() # key 값 확인
```

데이터 값 추출 : `dataset.key` or `dataset[key]`

- 예: 피쳐 데이터 값 추출 : `dataset.data` or `dataset['data']`

04. Model Selection Module

- 학습/테스트 데이터 세트 분리 - `train_test_split()`
 - 테스트 데이터 세트를 이용하지 않고 학습 데이터 세트로만 학습하고 예측하면 정확도 100%
 - Parameters of `train_test_split()`
 - `test_size` : 전체 데이터에서 테스트용 데이터 세트 크기를 얼마로 샘플링할 지
 - `train_size` : 전체 데이터에서 학습용 데이터 세트 크기를 얼마로 샘플링할 지
 - `shuffle` : 데이터를 분리하기 전에 데이터를 미리 섞을지 결정
 - `random_state` : 호출할 때마다 동일한 학습/테스트용 데이터 세트를 생성하기 위해 주어지는 난수 값
 - return tuple (학습용 데이터의 피쳐 데이터 세트, 테스트용 데이터의 피쳐 데이터 세트, 학습용 데이터의 레이블 데이터 세트, 테스트용 데이터의 레이블 데이터 세트)
- 교차 검증 (Cross-Validation)
 - 학습 데이터와 테스트용 데이터만으로는 과적합(Overfitting)에 취약하다.
 - 과적합
 - 모델이 학습 데이터에만 과도하게 최적화되어, 실제 예측을 다른 데이터로 수행할 경우 예측 성능이 떨어지는 것
 - 고정된 학습 데이터와 테스트 데이터로 평가를 하다 보면 테스트 데이터에만 최적의 성능을 발휘할 수 있도록 편향되게 모델을 유도해서 해당 테스트 데이터에만 과적합되는 학습 모델이 만들어져 다른 테스트용 데이터가 들어올 경우 성능이 저하됨
 - 이 문제점을 개선하기 위해 교차 검증을 이용함
 - 교차 검증 : 별도의 여러 세트로 구성된 학습 데이터 세트와 검증 데이터 세트에서 학습과 평가를 수행

- 그 각 세트에서 수행한 평가 결과에 따라 하이퍼 파라미터 튜닝 등의 모델 최적화도 손쉽게 할 수 있음
- ML 모델 성능 평가
 - 1단계 : 교차 검증 기반
 - 최종 단계 : 테스트 데이터 세트에 적용
 - 사용 되는 데이터 세트
 - 학습 데이터 세트(학습 데이터 세트 + 검증 데이터 세트) + 테스트 데이터 세트
- K 폴드 교차 검증
 - K개의 데이터 폴드 세트를 만들어서 K번 만큼 각 세트에 학습과 검증 평가를 반복적으로 수행하는 방법
 - 예:
 - 데이터 세트를 5등분하고 예측 평가를 구함
 - 첫 번째 반복 :
 - 학습 데이터 세트 : 2,3,4,5
 - 검증 데이터 세트 : 1
 - 두 번째 반복 :
 - 학습 데이터 세트 : 1,3,4,5
 - 검증 데이터 세트 : 2
 - 세 번째 반복 :
 - 학습 데이터 세트 : 1,2,4,5
 - 검증 데이터 세트 : 3
 - 네 번째 반복 :
 - 학습 데이터 세트 : 1,2,3,5
 - 검증 데이터 세트 : 4
 - 다섯 번째 반복 :
 - 학습 데이터 세트 : 1,2,3,4
 - 검증 데이터 세트 : 5

- K폴드 평가 결과 : 5개의 예측 평가를 평균
- 사이킷런에서는 K폴드 교차 검증 프로세스를 구현하기 위해 KFold와 StratifiedKFold 클래스를 제공
- KFold
 - KFold 객체 생성

```
kfold = KFold(n_splits=5)
```

- KFold 객체의 split()을 호출해 학습용/검증용 데이터로 분할

```
for train_index, test_index in kfold.split(features):
    ...
```

kfold.split(features) : 학습용/검증용 데이터로 분할할 수 있는 인덱스를 반환

```
[
  (
    array([30, 31, ..., 149]), #except 1st batch
    array([0,1,2,3,4,...,29]) #1st batch
  ),
  (
    array([1, 2, ..., 149]), #except 2nd batch
    array([30,1,2,3,4,...,59]) #2nd batch
  )
  (
    array([1, 2, ..., 149]), #except 3rd batch
    array([60,1,2,3,4,...,89]) #3rd batch
  ),
  (
    array([1, 2, ..., 149]), #except 4th batch
    array([90,1,2,3,4,...,119]) #4th batch
  )
  (
    array([1, 2, ..., 119]), #except 5th batch
    array([120,1,2,3,4,...,149]) #5th batch
  )
]
```

```
)  
]
```

- 학습 및 예측
- 반복 시마다 정확도 측정
- 개별 iteration별 정확도를 합하여 평균 정확도 계산

■ Stratified K 폴드

- 불균형한(imbalanced) 분포도를 가진 레이블 데이터 집합을 위한 방식
- 특정 레이블 값이 특이하게 많거나 매우 적어서 값의 분포가 한 쪽으로 치우치는 것
- 예:
 - 대출 사기 데이터 예측
 - 수십 개의 피처, 대출 사기 여부 레이블(대출 사기: 1, 정상 대출: 0)
 - 대부분의 데이터는 정상 대출일 거고 0,0001%만 대출 사기 레이블(1)로 존재
 - 작은 비율로 1 레이블 값이 있다면 K 폴드로 랜덤하게 학습 및 테스트 세트의 인덱스를 고르더라도 레이블 값인 0과 1의 비율을 제대로 반영 하지 못함
 - 원본 데이터와 유사한 대출 사기 레이블 값의 분포를 학습/테스트 세트에도 유지하는 게 매우 중요함
- Stratified K 폴드는 원본 데이터의 레이블 분포를 먼저 고려한 뒤 이 분포와 동일하게 학습과 검증 데이터 세트를 분배
- K 폴드의 문제 점

```
kfold = KFold(n_splits=3)  
for train_index, test_index in kfold.split(iris_df):  
    ...  
    print('학습 레이블 데이터 분포:\n',label_train.value_counts())  
    print('검증 레이블 데이터 분포:\n',label_test.value_counts())
```

- Output
 - 학습 레이블 데이터 분포:

2 50

1 50

검증 레이블 데이터 분포:

0 50

- 학습 레이블이 1,2 값이 각각 50개, 검증 레이블의 0값이 50개
 - 학습 레이블은 1,2밖에 없으므로 0의 경우를 학습하지 못함, 검증 레이블은 0밖에 없으므로 학습 모델은 절대 0을 예측하지 못하기 때문에 검증 예측 정확도는 0이 될 수 밖에 없음
- Stratified K 폴드 이용해서 문제 해결 가능 여부 확인
 - K 폴드와 유사 차이 하나 : split() 메서드에 인자로 피쳐 데이터 세트 뿐만 아니라 레이블 데이터 세트도 반드시 필요 (레이블 데이터 분포도에 따라 학습/검증 데이터를 나누기 때문)

```
skf = StratifiedKFold(n_splits=3)
for train_index, test_index in skf.split(iris_df, iris_df['label']):
    ...
    print('학습 레이블 데이터 분포:\n',label_train.value_counts())
    print('검증 레이블 데이터 분포:\n',label_test.value_counts())
```

- Output
 - 학습 레이블 데이터 분포:
0 34
1 33
2 33
검증 레이블 데이터 분포:
0 17
1 17
2 16
 - 학습 레이블과 검증 레이블 데이터 값의 분포도가 거의 동일하게 할당
 - 100개의 학습 레이블 : 각각 34,33,33 개
 - 50개의 검증 레이블 : 각각 17,17,16개

- Stratified K 폴드를 이용해 데이터를 교차 검증

- StratifiedKFold 객체 생성

```
skfold = StratifiedKFold(n_splits=3)
```

- skfold 객체의 split()을 호출해 학습용/검증용 데이터로 분할

```
for train_index, test_index in skfold.split(features,label):
    ...
```

- 학습 및 예측
- 반복 시마다 정확도 측정
- 개별 iteration별 정확도를 합하여 평균 정확도 계산

- 일반적으로 분류(Classification)에서 교차 검증은 Stratified K폴드로 분할하고, 회귀(Regression)에서는 Stratified K폴드가 지원 안 됨(회귀의 결정값은 이산값 형태의 레이블이 아니라 연속된 숫자값이기 때문에 결정값별로 분포를 정하는게 의미 없기 때문)
- 교차 검증을 간편하게 - cross_val_score()
 - KFold로 데이터를 학습하고 예측하는 코드:
 1. 폴드 세트를 설정
 2. for 루프에서 반복으로 학습 및 테스트 데이터의 인덱스를 추출
 3. 반복적으로 학습과 예측을 수행
 - 반환값 : 예측 성능
 - cross_val_score()는 위의 과정을 한꺼번에 수행해주는 API
 - cross_val_score(estimator, X, y=None, cv=None, n_jobs=1, verbose=0, fit_params=None, pre_dispatch='2*n_jobs')
 - 주요 파라미터 :
 - estimator : Classifier & Regressor
 - X : 피쳐 데이터셋
 - y : 레이블 데이터 세트
 - scoring : 예측 성능 평가 지표

- cv : 교차 검증 폴드 수
- 반환 값: scoring 파라미터로 지정된 성능 지표 측정값을 배열 형태로
 - 그것을 평균해서 평가 수치로 사용

◦ 예:

```
scores = cross_val_score(dt_clf, data, label, scoring='accuracy', cv=3)
```

- 비슷한 API cross_validate() : 단 하나의 평가 지표만 아니라 여러 개의 평가 지표를 반환
- GridSearchCV : 교차 검증과 최적 하이퍼 파라미터 튜닝을 한 번에
 - 하이퍼 파라미터 : 머신러닝 알고리즘의 예측 성능을 개선 할 수 있는 주요 구성 요소
 - 사이킷런은 GridSearchCV API를 이용해 하이퍼 파라미터를 순차적으로 입력하면서 편리하게 최적의 파라미터를 도출할 수 있는 방안을 제공
 - 파라미터 집합을 만들고 총 6회에 걸쳐 파라미터를 순차적으로 바꿔 실행하면서 최적의 파라미터와 수행 결과를 도출

```
grid_parameters = {'max_depth':[1,2,3], 'min_samples_split':[2,3]}
```

- 6회인 이유
 - CV가 3회이기 때문에 파라미터 조합마다 3개의 폴딩 세트를 3회에 걸쳐 학습/평가해 평균값으로 성능을 측정
 - 2개의 파라미터 조합('max_depth, 'min_samples_split') : 3회 x 2개 = 6회
- train_test_split()을 이용해 학습 데이터와 테스트 데이터를 먼저 분리하고 학습 데이터에서 GridSearchCV를 이용해 최적 하이퍼 파라미터를 추출

```
X_train, X_test, y_train, y_test = train_test_split(iris_data.data,
iris_data.target, test_size=0.2, random_state=121)
dtree = DecisionTreeClassifier()
parameters = {'max_depth':[1,2,3], 'min_samples_split':[2,3]}

grid_dtree = GridSearchCV(dtree, param_grid=parameters, cv=3, refit=
```

■ 주요 파라미터들

- estimator (dtree) : classifier, regressor, pipeline
- param_grid : key + 리스트 값을 가지는 딕셔너리
- scoring : 예측 성능을 측정할 평가 방법
- cv : 교차 검증을 위해 분할되는 학습/테스트 세트의 개수
- refit : 디폴트가 True, True로 생성 시 가장 최적의 하이퍼 파라미터를 찾은 뒤 입력된 estimator 객체를 해당 하이퍼 파라미터로 재학습

```
grid_dtree.fit(X_train, y_train)

scores_df = pd.DataFrame(grid_dtree.cv_results_)
scores_df[['params', 'mean_test_score', 'rank_test_score',
'split0_test_score', 'split1_test_score', 'split2_test_score']]

print(grid_dtree.best_params_)
print(grid_dtree.best_score_)
```

■ 학습 데이터 세트를 GridSearchCV 객체의 fit() 메서드에 인자로 받는다.

- fit() 수행시 학습 데이터를 cv에 기술된 폴딩 세트로 분할해 param_grid에 기술된 하이퍼 파라미터를 순차적으로 변경하면서 학습/평가를 수행
- 그 결과를 cv_results_속성에 기록
 - cv_results_ : 딕셔너리 형태로 key 값과 리스트 형태의 value 값을 가짐
- scores_df로 6개의 결과를 본다
 - params : 수행할 때마다 적용된 개별 하이퍼 파라미터 값
 - rank_test_score : 하이퍼 파라미터별로 성능이 좋은 score 순위
 - mean_test_score : 개별 하이퍼 파라미터별로 CV의 폴딩 테스트 세트에 대해 총 수행한 평가 평균값
 - split_test_score : 폴딩 세트들에서 각각 테스트한 성능 수치
 - 예 : CV가 3인 경우, 3개의 폴딩 세트에서 각각 테스트한 성능 수치

- GridSearchCV 객체의 fit()을 수행하면 최고 성능을 나타낸 하이퍼 파라미터의 값을 best_params_에 평가 결과 값을 best_score_ 속성에 기록 (cv_results의 rank_test_score가 1일 때의 값들)

```
estimator = grid_dtree.best_estimator_
```

```
pred = estimator.predict(X_test)
print(accuracy_score(y_test, pred))
```

- refit=True이면 GridSearchCV가 최적 성능을 나타내는 하이퍼 파라미터로 Estimator를 학습해 best_estimator_로 저장
- 이미 학습된 best_estimator_를 이용해 앞에서 train_test_split()으로 분리한 테스트 데이터 세트에 대해 예측하고 성능을 평가

05. 데이터 전처리

- 데이터 전처리는 머신러닝(ML) 알고리즘만큼 중요한 과정입니다. ML 알고리즘은 입력 데이터에 따라 성능이 달라지므로, 데이터 품질이 높아야 합니다(Garbage In, Garbage Out 원칙).
- 결손값(Null 값) 처리
 - 머신러닝 모델은 NaN, NULL 값을 허용하지 않으므로, 반드시 다른 값으로 변환해야 합니다.
 - 해결 방법
 - 결손값이 적다면 평균값, 최빈값, 중위값으로 대체
 - 결손값이 많다면 해당 컬럼(피처) 삭제
 - 중요한 피처라면 비즈니스 로직에 맞는 값으로 변환
- 데이터 인코딩
 - 사이킷런의 ML 알고리즘은 문자열 값을 입력값으로 허용하지 않음 → 모든 문자열을 숫자로 변환(인코딩)해야 함
- 레이블 인코딩(Label Encoding)
 - 카테고리 데이터를 순서가 있는 숫자 값으로 변환
 - 문제점: 숫자가 크기 개념을 가지므로 일부 ML 알고리즘에서 왜곡 발생 가능
 - 트리 계열 알고리즘에서는 문제 없음


```

from sklearn.preprocessing import LabelEncoder

items = ['TV', '냉장고', '전자레인지', '컴퓨터', '선풍기', '믹서']

# LabelEncoder 객체 생성 및 변환
encoder = LabelEncoder()
encoder.fit(items)
labels = encoder.transform(items)

print("레이블 인코딩 결과:", labels)
print("인코딩 클래스:", encoder.classes_)

# 디코딩
decoded_labels = encoder.inverse_transform(labels)
print("디코딩된 원본 값:", decoded_labels)

```

- 원-핫 인코딩(One-Hot Encoding)
 - 레이블 인코딩 문제 해결 → 숫자 크기 의미 없도록 각 카테고리를 새로운 컬럼으로 변환
 - 고유한 카테고리별로 0 또는 1 값만 저장

```

from sklearn.preprocessing import OneHotEncoder
import numpy as np

# 데이터 준비 (2D 배열로 변환 필수)
items = np.array(['TV', '냉장고', '전자레인지', '컴퓨터', '선풍기', '믹서']).
reshape(-1, 1)

# OneHotEncoder 적용
oh_encoder = OneHotEncoder()
oh_encoder.fit(items)
oh_labels = oh_encoder.transform(items)

# 변환된 원-핫 인코딩 데이터 출력
print("원-핫 인코딩 결과:\n", oh_labels.toarray())
print("원-핫 인코딩 데이터 차원:", oh_labels.shape)

```

- 피처 스케일링과 정규화
 - 피처 스케일링(feature scaling)*은 서로 다른 변수의 값 범위를 일정한 수준으로 맞추는 작업.
 - 대표적인 방법으로 표준화(Standardization)와 정규화(Normalization)
- 표준화 (Standardization)
 - 평균이 0, 분산이 1이 되도록 데이터를 변환하는 과정
 - 가우시안 정규 분포(정규 분포)를 가정하는 알고리즘(SVM, 선형 회귀, 로지스틱 회귀 등)에서 유용함
 - 변환 공식:

$$xi_new = xi - \text{mean}(x) / \text{stdev}(x)$$

```
from sklearn.preprocessing import StandardScaler
import pandas as pd
from sklearn.datasets import load_iris

iris = load_iris()
iris_df = pd.DataFrame(data=iris.data, columns=iris.feature_names)

scaler = StandardScaler()
iris_scaled = scaler.fit_transform(iris_df)

iris_df_scaled = pd.DataFrame(data=iris_scaled, columns=iris.feature_names)
print(iris_df_scaled.mean()) # 평균이 0에 가까운지 확인
print(iris_df_scaled.var()) # 분산이 1에 가까운지 확인
```

- 정규화 (Normalization)
 - 데이터의 최소값과 최대값을 0~1(- 이라면 -1~1)범위로 변환하는 과정
 - 데이터의 분포가 정규 분포가 아닐 경우에 유용함
 - 변환 공식:

$$xi_new = xi - \min(x) / \max(x) - \min(x)$$

```
from sklearn.preprocessing import MinMaxScaler
```

```

scaler = MinMaxScaler()
iris_scaled = scaler.fit_transform(iris_df)

iris_df_scaled = pd.DataFrame(data=iris_scaled, columns=iris.feature_names)
print(iris_df_scaled.min()) # 0에 가까운 값
print(iris_df_scaled.max()) # 1에 가까운 값

```

- 벡터 정규화 (Vector Normalization)

- 변환 공식

$$xi_new = xi / \sqrt{xi^2 + yi^2 + zi^2}$$

- StandardScaler

- 평균이 0, 분산이 1이 되도록 데이터를 변환하는 클래스
 - 주로 SVM(Support Vector Machine), 선형 회귀(Linear Regression), 로지스틱 회귀(Logistic Regression) 모델에서 사용

```

from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
scaler.fit(iris_df)
iris_scaled = scaler.transform(iris_df)

```

- MinMaxScaler

- 0과 1 사이의 값으로 데이터를 변환하는 클래스 (음수 값이 있으면 -1~1 사이로 변환)
 - 가우시안 분포가 아닌 경우에 사용 가능

```

from sklearn.preprocessing import MinMaxScaler
scaler = MinMaxScaler()
scaler.fit(iris_df)
iris_scaled = scaler.transform(iris_df)

```

- 학습 데이터와 테스트 데이터 스케일링 주의점

- 학습 데이터로 fit() 을 한 후 테스트 데이터에는 같은 변환 기준을 적용해야 합니다.

- 테스트 데이터에 다시 fit() 을 적용하면 잘못된 변환이 발생할 수 있습니다.
- 잘못된 예(테스트 데이터에 fit 적용):

```
train_array = np.arange(0, 11).reshape(-1, 1)
test_array = np.arange(0, 6).reshape(-1, 1)

scaler = MinMaxScaler()
scaler.fit(train_array) # 학습 데이터로 fit
train_scaled = scaler.transform(train_array)

scaler.fit(test_array) # 테스트 데이터에 다시 fit (잘못된 방법)
test_scaled = scaler.transform(test_array)
```

- 올바른 예(학습 데이터로 fit 후 테스트 데이터 변환):

```
scaler = MinMaxScaler()
scaler.fit(train_array) # 학습 데이터에 fit
train_scaled = scaler.transform(train_array)
test_scaled = scaler.transform(test_array) # 학습 데이터의 기준을 적용
```

평가 (Evaluation)

01. 정확도(Accuracy)

- 머신러닝 평가 프로세스: 머신러닝은 데이터 가공, 모델 학습/예측, 평가로 구성됩니다. 평가는 모델의 예측 성능을 측정하는 중요한 단계입니다.
- 평가 지표:
 - 정확도(Accuracy): 예측값과 실제값이 일치하는 비율. 하지만 불균형 데이터셋에서는 정확도만으로 모델 성능을 평가하기 어렵습니다.
 - 오차행렬(Confusion Matrix): 예측값과 실제값의 관계를 TN, FP, FN, TP로 나누어 모델의 오류 유형을 분석합니다.
 - 정밀도(Precision): Positive로 예측한 값 중 실제 Positive인 비율. FP를 줄이는데 초점을 맞춥니다.
 - 재현율(Recall): 실제 Positive인 값 중 예측이 Positive인 비율. FN을 줄이는데 초점을 맞춥니다.

- F1 스코어: 정밀도와 재현율의 조화평균으로, 두 지표를 균형 있게 평가합니다.
- ROC AUC: 모델의 분류 성능을 종합적으로 평가하는 지표입니다.
- 이진 분류와 멀티 분류: 분류 문제는 이진 분류(두 개의 클래스)와 멀티 분류(여러 클래스)로 나뉩니다. 이진 분류에서는 정밀도와 재현율이 특히 중요합니다.
- 불균형 데이터셋의 문제: 데이터셋의 클래스 분포가 불균형할 경우, 정확도만으로는 모델의 성능을 제대로 평가할 수 없습니다. 예를 들어, 모든 예측을 Negative로 하더라도 정확도가 높게 나올 수 있습니다.
- 정밀도와 재현율의 중요성:
 - 재현율: 실제 Positive인 경우를 놓치지 않는 것이 중요한 경우(예: 암 진단, 금융 사기 탐지)에 중요합니다.
 - 정밀도: Negative를 Positive로 잘못 예측할 경우 문제가 되는 경우(예: 스팸 메일 탐지)에 중요합니다.

02. 오차 행렬

- 오차행렬의 활용: 오차행렬을 통해 TN, FP, FN, TP를 분석하여 모델의 성능을 세부적으로 평가할 수 있습니다. 이를 통해 정밀도와 재현율을 계산하고, 모델의 강점과 약점을 파악할 수 있습니다.
- 예제: 타이타닉 생존자 예측과 MNIST 데이터셋을 통해 불균형 데이터셋에서 정확도의 한계를 보여주고, 정밀도와 재현율의 중요성을 강조합니다.
- 결론적으로, 머신러닝 모델의 성능을 평가할 때는 정확도뿐만 아니라 정밀도, 재현율, F1 스코어, ROC AUC 등 다양한 지표를 종합적으로 고려해야 하며, 특히 불균형 데이터셋에서는 정밀도와 재현율이 중요한 평가 기준이 됩니다.
- 평가 지표 계산 함수 생성
- `get_clf_eval()` 함수: 오차 행렬(Confusion Matrix), 정확도(Accuracy), 정밀도(Precision), 재현율(Recall)을 한 번에 계산하는 함수입니다.

```
from sklearn.metrics import accuracy_score, precision_score, recall_score,
confusion_matrix
```

```
def get_clf_eval(y_test, pred):
    confusion = confusion_matrix(y_test, pred)
    accuracy = accuracy_score(y_test, pred)
    precision = precision_score(y_test, pred)
    recall = recall_score(y_test, pred)
```

```
print('오차 행렬')
print(confusion)
print('정확도: {0:.4f}, 정밀도: {1:.4f}, 재현율: {2:.4f}'.
      format(accuracy, precision, recall))
```

- 로지스틱 회귀 모델 적용
- 타이타닉 데이터셋을 로지스틱 회귀 모델로 학습하고, `get_clf_eval()` 함수를 사용하여 성능을 평가합니다.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression

# 데이터 로드 및 전처리
titanic_df = pd.read_csv('./titanic_train.csv')
y_titanic_df = titanic_df['Survived']
X_titanic_df = titanic_df.drop('Survived', axis=1)
X_titanic_df = transform_features(X_titanic_df)
X_train, X_test, y_train, y_test = train_test_split(X_titanic_df,
                                                    y_titanic_df, test_size=0.2, random_state=11)

# 로지스틱 회귀 모델 학습 및 예측
lr_clf = LogisticRegression(solver='liblinear')
lr_clf.fit(X_train, y_train)
pred = lr_clf.predict(X_test)
get_clf_eval(y_test, pred)
```

- 임계값 조정을 통한 정밀도/재현율 트레이드오프
 - `predict_proba()`: 각 클래스별 예측 확률을 반환합니다.
 - `Binarizer`: 임계값을 기준으로 Positive/Negative를 결정합니다.

```
from sklearn.preprocessing import Binarizer

# 임계값 0.5로 설정
custom_threshold = 0.5
pred_proba_1 = pred_proba[:, 1].reshape(-1, 1)
binarizer = Binarizer(threshold=custom_threshold).fit(pred_proba_1)
```

```
custom_predict = binarizer.transform(pred_proba_1)
get_clf_eval(y_test, custom_predict)
```

- 임계값을 0.4로 낮춘 경우:

출력 결과:

- 재현율이 증가하지만, 정밀도가 감소합니다.

```
custom_threshold = 0.4
binarizer = Binarizer(threshold=custom_threshold).fit(pred_proba_1)
custom_predict = binarizer.transform(pred_proba_1)
get_clf_eval(y_test, custom_predict)
```

오차 행렬

[[97 21]

[11 50]]

정확도: 0.8212, 정밀도: 0.7042, 재현율: 0.8197

- 임계값에 따른 평가 지표 변화
- 다양한 임계값에 대해 정확도, 정밀도, 재현율을 계산합니다.

```
thresholds = [0.4, 0.45, 0.50, 0.55, 0.60]
```

```
def get_eval_by_threshold(y_test, pred_proba_c1, thresholds):
    for custom_threshold in thresholds:
        binarizer = Binarizer(threshold=custom_threshold).fit(pred_proba_c1)
        custom_predict = binarizer.transform(pred_proba_c1)
        print('임계값:', custom_threshold)
        get_clf_eval(y_test, custom_predict)

get_eval_by_threshold(y_test, pred_proba[:, 1].reshape(-1, 1), thresholds)
```

출력 결과:

임계값: 0.4

정확도: 0.8212, 정밀도: 0.7042, 재현율: 0.8197

임계값: 0.45

정확도: 0.8547, 정밀도: 0.7869, 재현율: 0.7869

임계값: 0.5
정확도: 0.8659, 정밀도: 0.8246, 재현율: 0.7705
임계값: 0.55
정확도: 0.8715, 정밀도: 0.8654, 재현율: 0.7377
임계값: 0.6
정확도: 0.8771, 정밀도: 0.8980, 재현율: 0.7213

03. 정밀도-재현율

- `precision_recall_curve()` 를 사용하여 정밀도와 재현율의 관계를 시각화합니다.

```
from sklearn.metrics import precision_recall_curve
import matplotlib.pyplot as plt

def precision_recall_curve_plot(y_test, pred_proba_c1):
    precisions, recalls, thresholds = precision_recall_curve(y_test,
pred_proba_c1)
    plt.figure(figsize=(8, 6))
    plt.plot(thresholds, precisions[:-1], linestyle='--', label='precision')
    plt.plot(thresholds, recalls[:-1], label='recall')
    plt.xlabel('Threshold value')
    plt.ylabel('Precision and Recall value')
    plt.legend()
    plt.grid()
    plt.show()

precision_recall_curve_plot(y_test, lr_clf.predict_proba(X_test)[: , 1])
```

- 정밀도와 재현율의 극단적 조정
 - 정밀도 100%: 확실한 Positive만 예측 (FP = 0).
 - 재현율 100%: 모든 데이터를 Positive로 예측 (FN = 0).
 - 이러한 극단적인 조정은 모델의 실용성을 떨어뜨리므로, 두 지표를 균형 있게 고려해야 합니다.

04. F1 스코어

- 정밀도(Precision)와 재현율(Recall)을 결합한 지표
- 공식:

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

- 정밀도와 재현율이 균형을 이룰 때 높은 값을 가짐
- 예제:
 - A 모델(Precision: 0.9, Recall: 0.1) → **F1: 0.18**
 - B 모델(Precision: 0.5, Recall: 0.5) → **F1: 0.5** (더 우수함)

05. ROC 곡선과 AUC

- ROC 곡선 (Receiver Operating Characteristic Curve)
 - X축: FPR (False Positive Rate)
 - Y축: TPR (True Positive Rate, 즉 Recall)
 - FPR이 변할 때 TPR이 어떻게 변하는지 시각화
 - 분류 모델의 성능을 평가하는 중요한 지표
- AUC (Area Under Curve)
 - ROC 곡선 아래 면적
 - 값이 1에 가까울수록 좋은 모델
 - 랜덤 모델의 AUC 값은 0.5
- ROC 곡선 그리기 (사이킷런 사용)

```
from sklearn.metrics import roc_curve
fprs, tprs, thresholds = roc_curve(y_test, pred_proba)
```

- AUC 값 계산 (사이킷런 사용)

```
from sklearn.metrics import roc_auc_score
auc_score = roc_auc_score(y_test, pred_proba)
```

- F1 스코어 vs. ROC-AUC

지표	특징
F1 스코어	정밀도와 재현율의 조화 평균, 불균형 데이터에서 유용
ROC-AUC	분류 성능을 전체적으로 평가, 임계값 변화에 따른 성능 확인

